

EE2213 Introduction to AI

Lecture 11: Neural Networks

Dr. WANG Si

si.wang@nus.edu.sg

Electrical and Computer Engineering Department
National University of Singapore

OVERVIEW OF COURSE CONTENTS

- **Introduction (Shaojing)**

- What is AI
- Applications of AI
- AI agent

- **Search (Shaojing)**

- Uninformed search algorithms: breadth-first, depth-first, uniform-cost(Dijkstra's algorithm)
- Informed search algorithms: greedy best-first, A*
- Applications

- **Optimisation (Shaojing)**

- Linear programming
- Convex problems
- Applications

- **Machine learning (Wang Si) (Weeks 6-9)**

- Supervised and unsupervised learning: regression, classification, clustering
- Neural networks and deep learning
- Applications

- **Knowledge representation (Wang Si) (Weeks 10-11)**

- Knowledge Representation and Reasoning
- Propositional Logic
- Applications

- **Ethical considerations (Shaojing)**

- Bias in AI
- Privacy concerns
- Societal impact

(Week 10, Monday)

**No Class on 20 Oct.! A recording
will be uploaded on Canvas.**

Gentle Reminder

✓ 3rd Quiz on 17 Oct. (Week 9, Friday)

Quiz Day Schedule

- 12:00–12:30pm – Q&A
- 12:30–1:00pm – Break and preparation; Exit and re-enter LT7 with ID check
- 1:00–1:30pm – Quiz

Question Types and Marks Breakdown

- T/F : **2** questions × 2 marks
- MCQ (only one correct answer): **2** questions × 3 marks
- MRQ (allow multiple correct answers): **5** questions × 5 marks
- Fill-in-the-Blank Questions (FITB): **3** questions × 7 marks
- Total: **12** questions, 56 marks

AGENDA

- We will discuss:
 - Perceptron
 - Multilayer Perceptron (MLP)
 - MLPs for Image Classification

At the end of this lecture, you should be able to:

- ✓ Describe perceptron model
- ✓ Explain the architecture and learning process of MLP
- ✓ Apply MLP for image classification and identify the limitations

Recap: AI & ML & DL

ARTIFICIAL INTELLIGENCE

Any technique which enables computers to mimic human behavior



MACHINE LEARNING

AI techniques that give computers the ability to learn without being explicitly programmed to do so



DEEP LEARNING

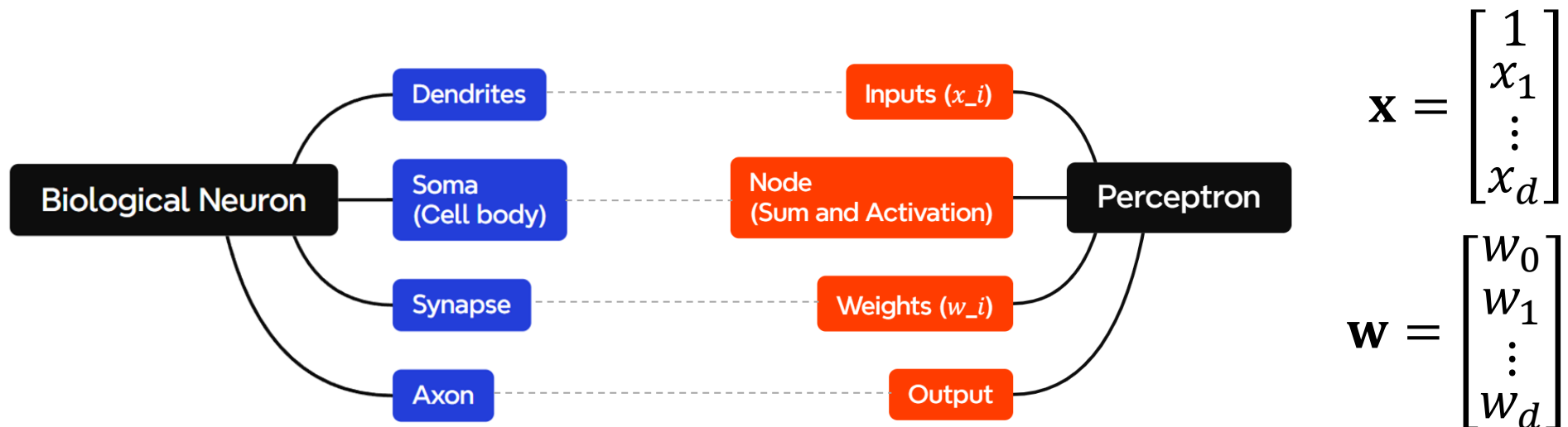
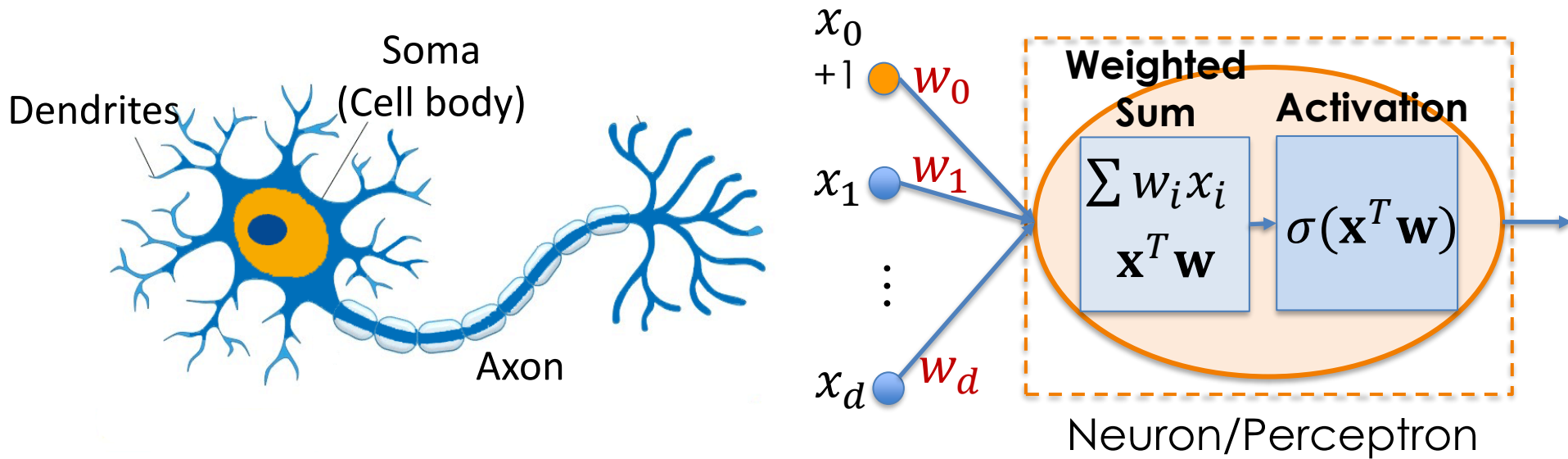
A subset of ML which make the computation of multi-layer neural networks feasible



1950's 1960's 1970's 1980's 1990's 2000's 2010s

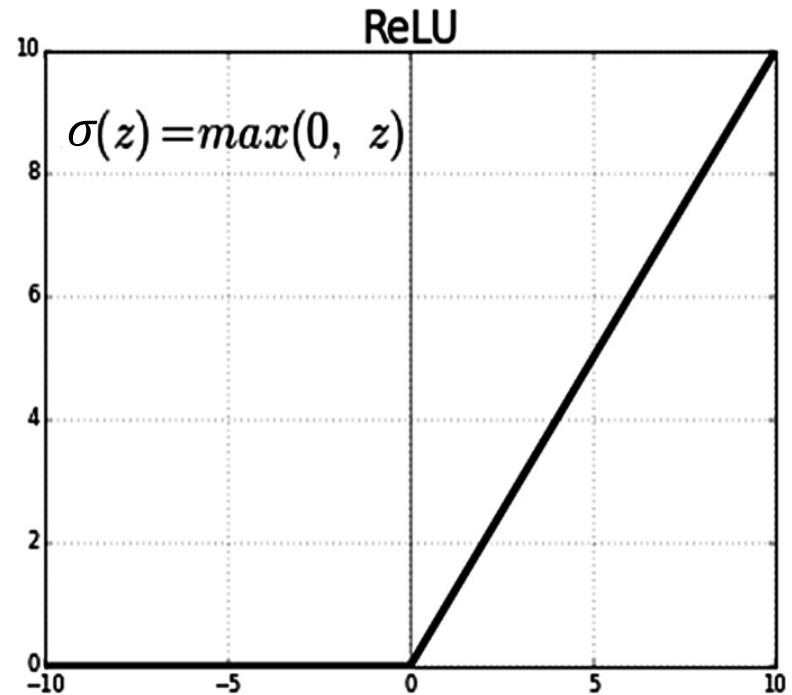
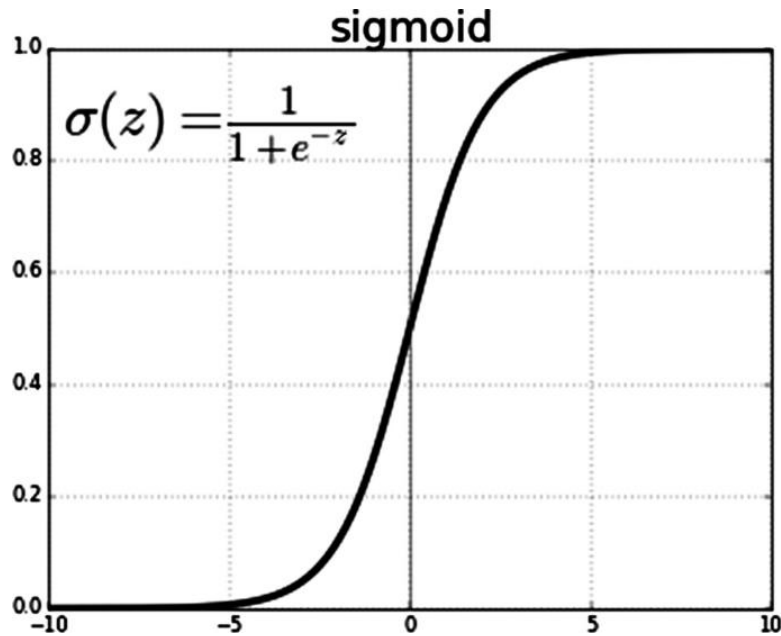
Perceptron

Biological Neuron & Perceptron

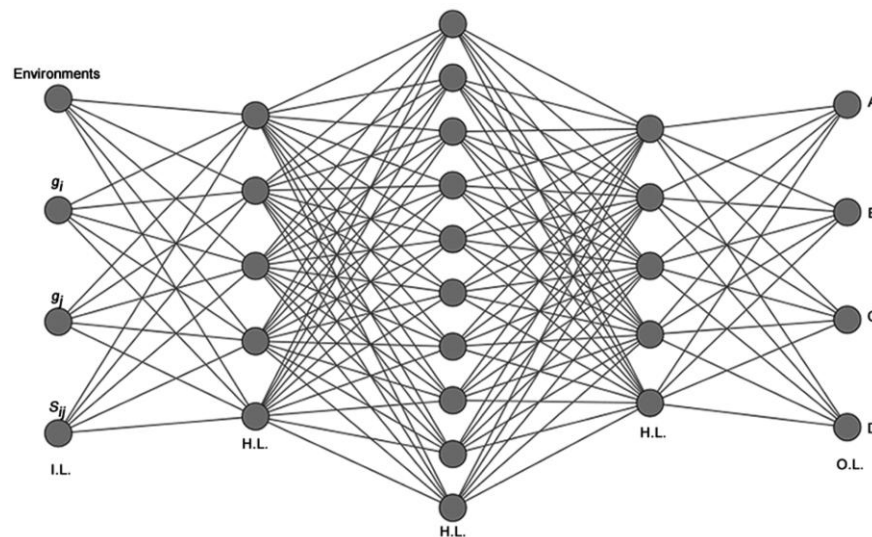


Perceptron

➤ Activation Function (Non-Linear)

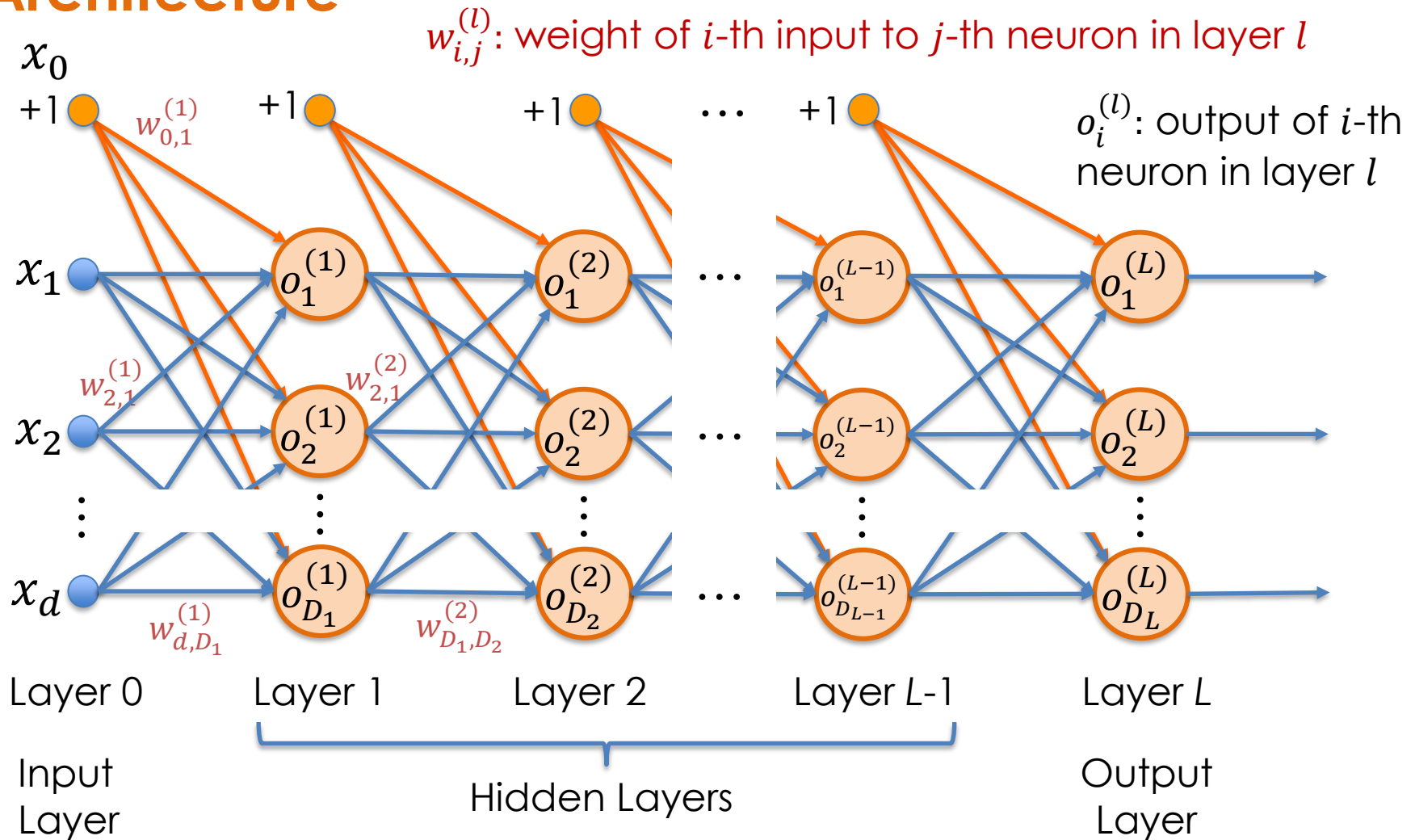


Multilayer Perceptron (MLP)



Multilayer Perceptron (MLP)

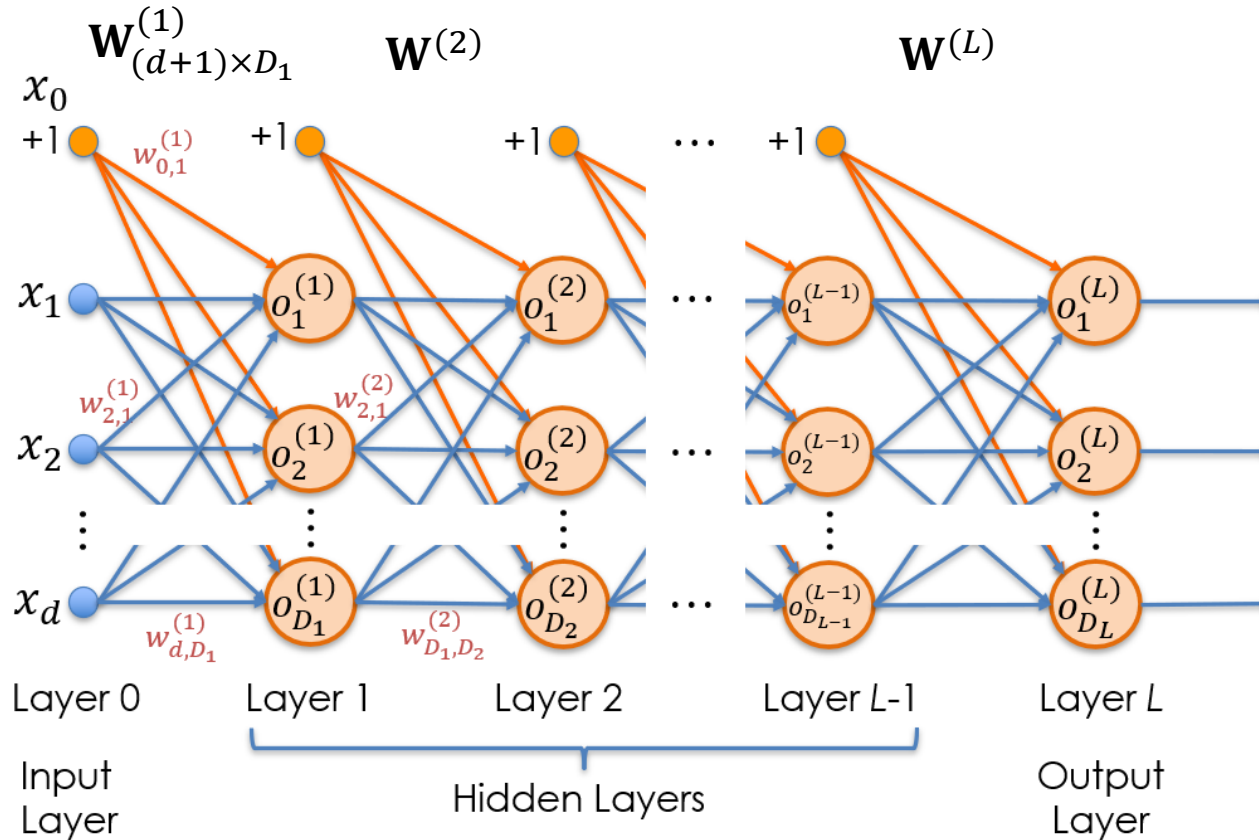
➤ Architecture



Note: D_l denotes the number of neurons in layer l

Multilayer Perceptron (MLP)

➤ Architecture



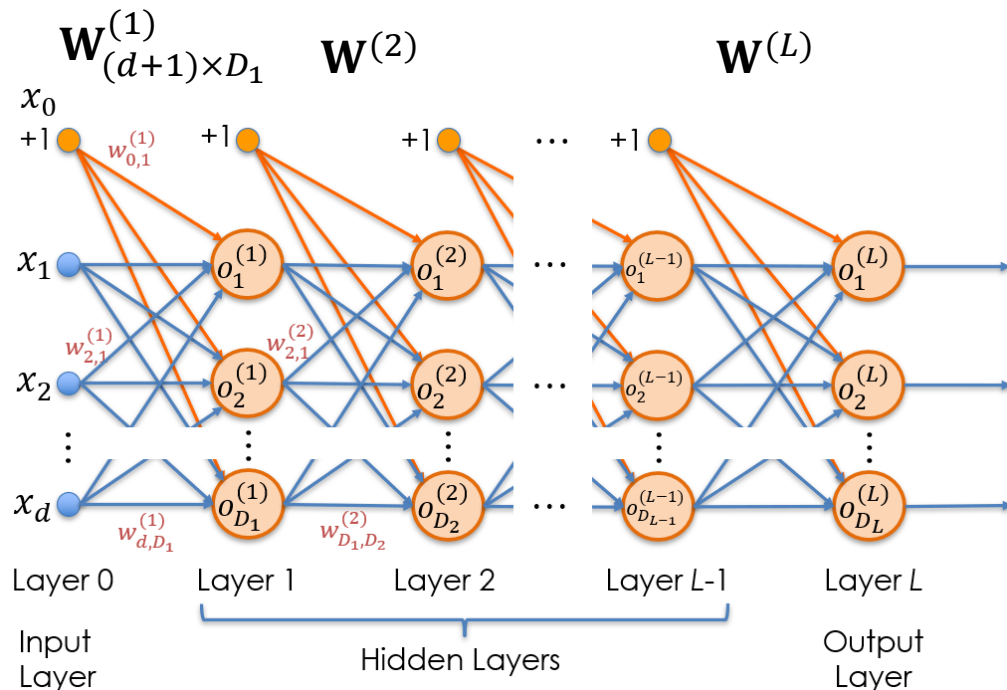
e.g., 3 inputs to layer 3,
2 neurons in layer 3

$$\mathbf{W}^{(l)} = \begin{bmatrix} w_{0,1}^{(l)} & \dots & w_{0,D_l}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{D_{l-1},1}^{(l)} & \dots & w_{D_{l-1},D_l}^{(l)} \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^{(l)} & \dots & \mathbf{w}_{D_l}^{(l)} \end{bmatrix}$$

↓
Weight vector for first neuron in layer l

Multilayer Perceptron (MLP)

➤ Architecture



$$\mathbf{W}^{(l)} = \begin{bmatrix} w_{0,1}^{(l)} & \dots & w_{0,D_l}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{D_{l-1},1}^{(l)} & \dots & w_{D_{l-1},D_l}^{(l)} \end{bmatrix}$$

$$= \begin{bmatrix} \mathbf{w}_1^{(l)} & \dots & \mathbf{w}_{D_l}^{(l)} \end{bmatrix}$$

Weight vector for first neuron in layer l

$$o_i^{(l)} = \sigma^{(l)} \left(\sum_{j=1}^{D_{l-1}} o_j^{(l-1)} w_{j,i}^{(l)} + w_{0,i}^{(l)} \right) = \sigma^{(l)} ([1, \mathbf{o}^{(l-1)}] \mathbf{w}_i^{(l)})$$

$$\mathbf{o}^{(l)} = \sigma^{(l)} ([1, \mathbf{o}^{(l-1)}] \mathbf{W}^{(l)})$$

A **MLP** is essentially a **nested function**

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \sigma^{(L)} ([1, \dots \sigma^{(2)} ([1, \sigma^{(1)} (\mathbf{x}^T \mathbf{W}^{(1)})] \mathbf{W}^{(2)}) \dots] \mathbf{W}^{(L)})$$

Multilayer Perceptron (MLP)

➤ Architecture

Example: A MLP of 2 layers has been constructed as:

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \sigma([1, \sigma(\mathbf{x}^T \mathbf{W}^{(1)})] \mathbf{W}^{(2)}),$$

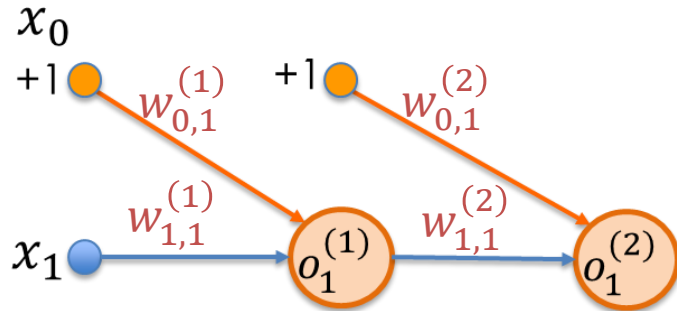
$$\text{where } \mathbf{x} = [1, 1, 3]^T, \mathbf{W}^{(1)} = \mathbf{W}^{(2)} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}, \sigma(z) = \max(0, z).$$

Compute network output $\mathbf{f}_{\mathbf{W}}(\mathbf{x})$.

Multilayer Perceptron (MLP)

➤ Architecture

Why non-linear activation function $\sigma(z)$?



$$o_1^{(2)} = \sigma(w_{0,1}^{(2)} + w_{1,1}^{(2)} o_1^{(1)})$$

$$= a(w_{0,1}^{(2)} + w_{1,1}^{(2)} o_1^{(1)}) + b$$

$$= (aw_{0,1}^{(2)} + b) + aw_{1,1}^{(2)} o_1^{(1)}$$

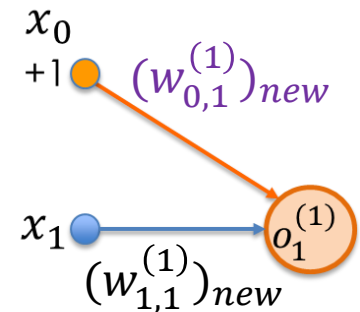
$$= (aw_{0,1}^{(2)} + b) + aw_{1,1}^{(2)} o_1^{(1)}$$

$$= (aw_{0,1}^{(2)} + b) + aw_{1,1}^{(2)} ((aw_{0,1}^{(1)} + b) + aw_{1,1}^{(1)} x_1)$$

$$= (aw_{0,1}^{(2)} + a^2 w_{1,1}^{(2)} w_{0,1}^{(1)} + ab w_{1,1}^{(2)} + b)$$

$$+ a^2 w_{1,1}^{(2)} w_{1,1}^{(1)} x_1$$

$$(w_{1,1}^{(1)})_{new}$$



If linear/affine activation function:

$$\sigma(z) = az + b$$

$$o_1^{(1)} = \sigma(w_{0,1}^{(1)} + w_{1,1}^{(1)} x_1)$$

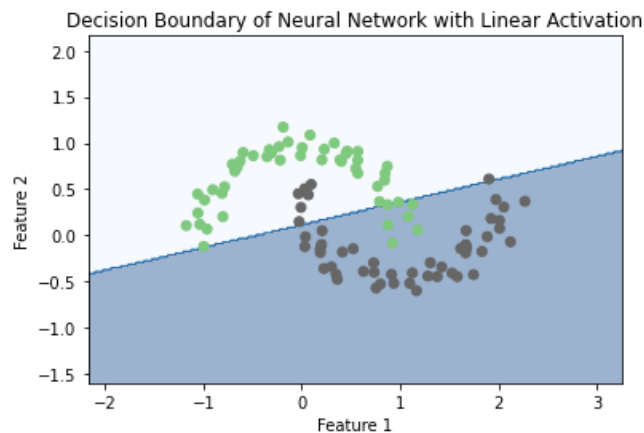
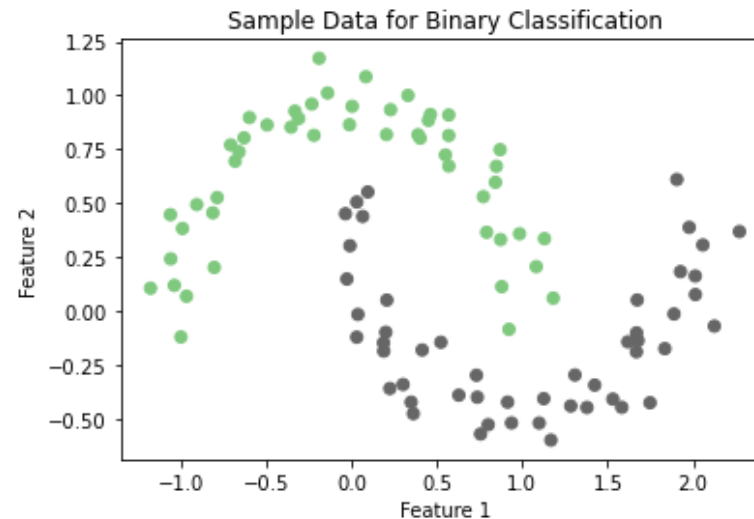
$$= a(w_{0,1}^{(1)} + w_{1,1}^{(1)} x_1) + b$$

$$= (aw_{0,1}^{(1)} + b) + aw_{1,1}^{(1)} x_1$$

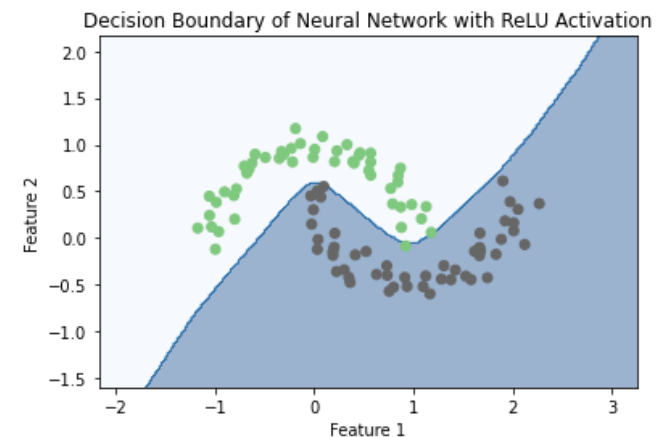
With Linear/affine $\sigma(z)$, MLP=Single-layer Perceptron!

Multilayer Perceptron (MLP)

➤ Visualization of Decision Boundaries



Linear/Affine Activation



ReLU Activation

Multilayer Perceptron (MLP)

➤ Learning (W)

1. Random Initialization of **W**

2. Forward Pass (fixed **W**)

i. Compute output responses

3. **Backward Pass** (update **W**)

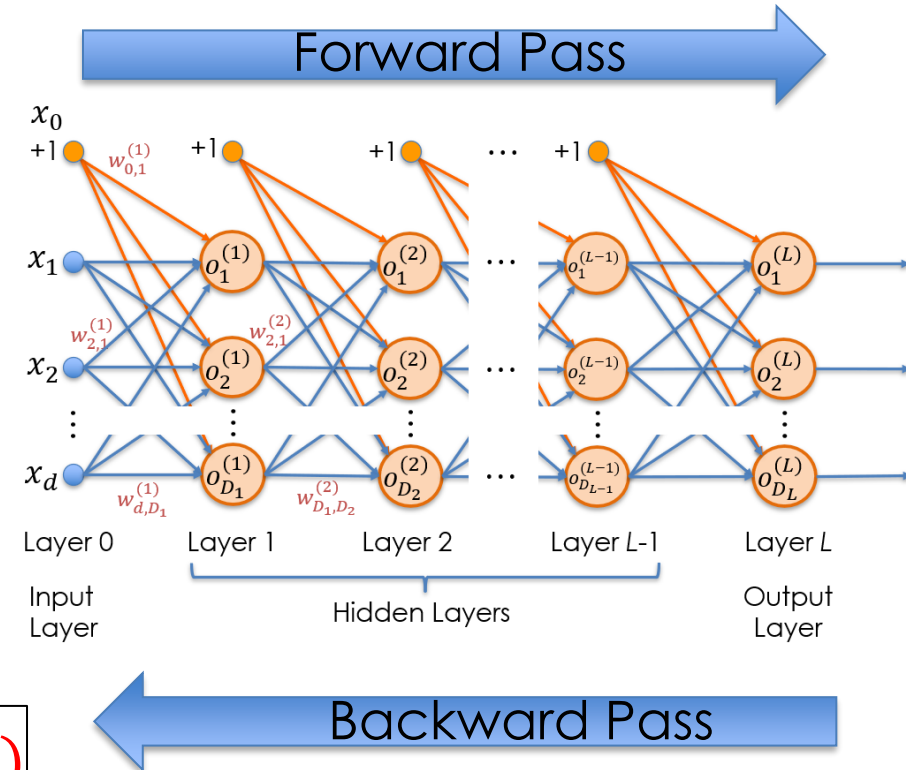
i.e., **Backpropagation**

i. Propagate the errors backward

ii. Update **W**

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \nabla_{\mathbf{W}} J(\mathbf{W}_{old})$$

In essence, Backpropagation= Gradient Decent!



Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

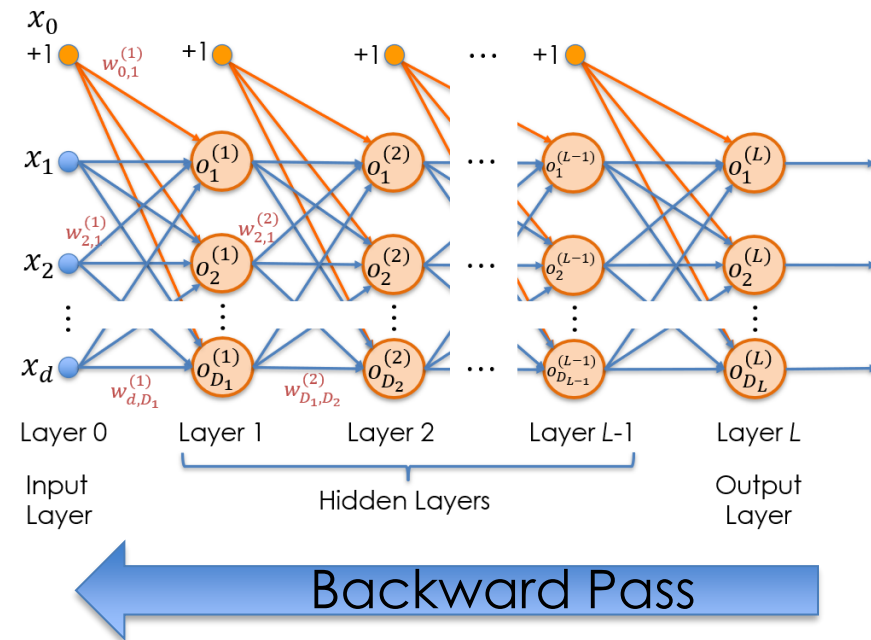
$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \nabla_{\mathbf{W}} J(\mathbf{W}_{old})$$

$$\nabla_{\mathbf{W}} J(\mathbf{W}_{old}) = \frac{1}{N} \sum \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W}_{old})$$

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}}$$

$$\text{where } z_j^{(l)} = (\mathbf{a}^{(l)})^T \mathbf{w}_j^{(l)}$$

$$\mathbf{a}^l: \text{input vector to layer } l = \begin{cases} \mathbf{x}, & \text{if } l = 1 \\ [1, \mathbf{o}^{(l-1)}]^T, & \text{else} \end{cases}$$



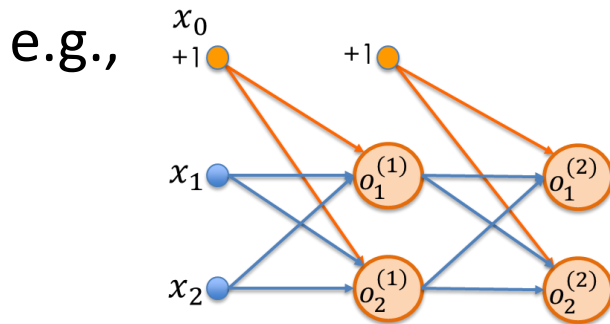
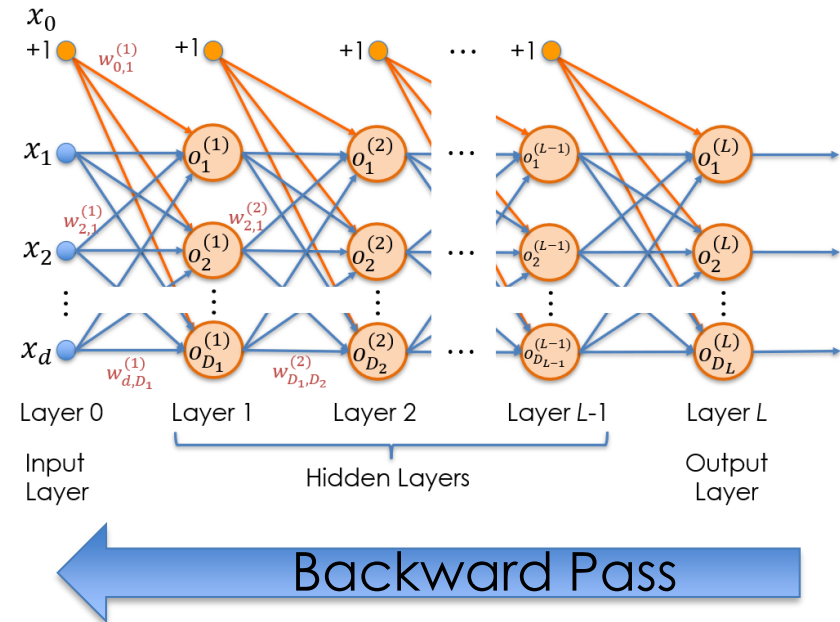
Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}}$$

$$\frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}} = \frac{\partial}{\partial w_{i,j}^{(l)}} \left(\sum_m w_{m,j}^{(l)} a_m^{(l)} \right)$$

$$= a_i^{(l)}$$



$$\frac{\partial z_1^{(2)}}{\partial w_{2,1}^{(2)}} = ?$$

$$\frac{\partial z_1^{(1)}}{\partial w_{0,1}^{(1)}} = ?$$

Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

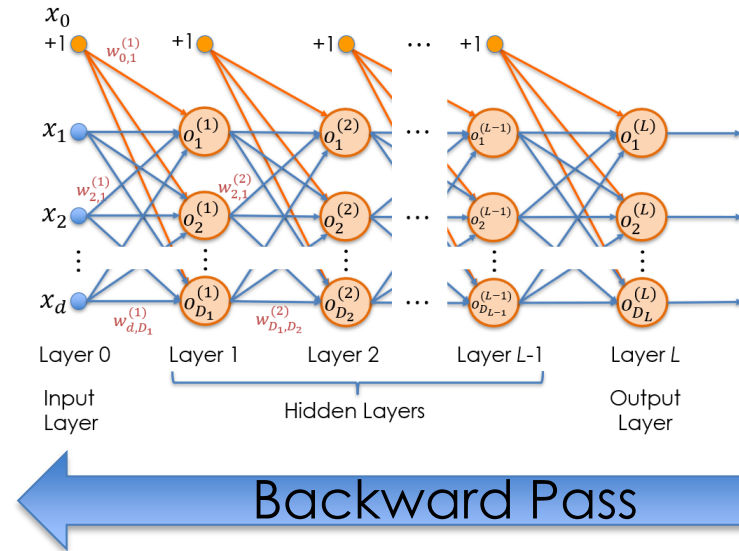
$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}}$$

$\epsilon_j^{(l)}$ (red box) $a_i^{(l)}$ (blue box)

- If output layer ($l=L$):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z_j^{(L)}} &= \frac{\partial \mathcal{L}(o_1^{(L)}, \dots, o_{D_L}^{(L)})}{\partial z_j^{(L)}} = \sum_{m=1}^{D_L} \frac{\partial \mathcal{L}(o_m^{(L)})}{\partial z_j^{(L)}} \\ &= \sum_{m=1}^{D_L} \frac{\partial \mathcal{L}(o_m^{(L)})}{\partial o_m^{(L)}} \frac{\partial o_m^{(L)}}{\partial z_j^{(L)}} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)}) \end{aligned}$$

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)}) a_i^{(l)}$$



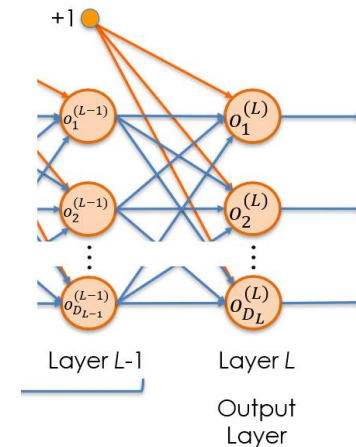
Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

If output layer ($l=L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(L)}} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)}) a_i^{(L)}$$

e.g., if $\underbrace{\sigma^{(L)}(\mathbf{z})_k = \frac{e^{z_k^{(L)}}}{\sum_{s=1}^{D_L} e^{z_s^{(L)}}}_{\text{Softmax}}, \mathcal{L}(\mathbf{W}) = - \underbrace{\sum_{k=1}^{D_L} y_k \ln(p_k)}_{\text{Categorical Cross Entropy}}. \text{ What is } \frac{\partial \mathcal{L}}{\partial w_{i,j}^{(L)}}?$



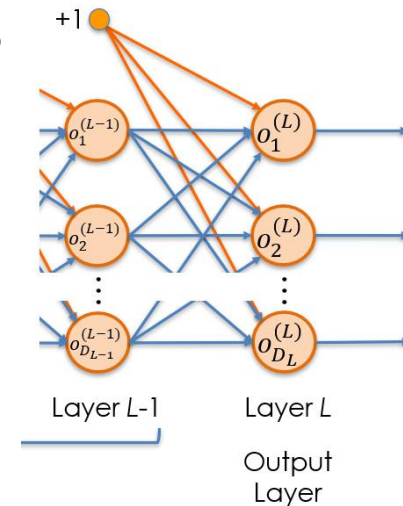
Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

If output layer ($l=L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(L)}} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)}) a_i^{(L)}$$

e.g., if $\underbrace{\sigma^{(L)}(\mathbf{z})_k = \frac{e^{z_k^{(L)}}}{\sum_{s=1}^{D_L} e^{z_s^{(L)}}}_{\text{Softmax}}, \mathcal{L}(\mathbf{W}) = - \underbrace{\sum_{k=1}^{D_L} y_k \ln(p_k)}_{\text{Categorical Cross Entropy}}. \text{ What is } \frac{d\mathcal{L}}{d\mathbf{W}^{(L)}}?$



Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

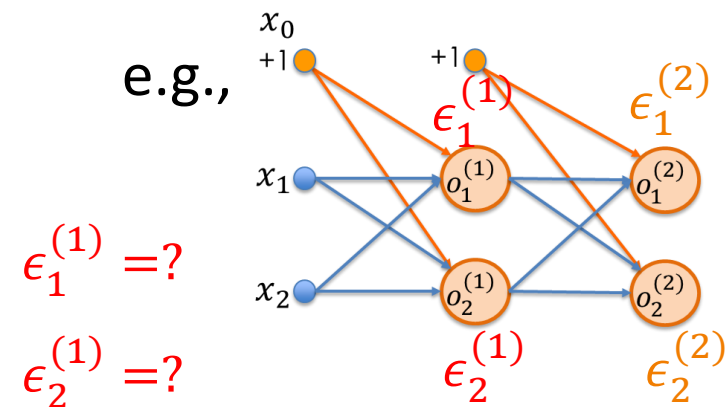
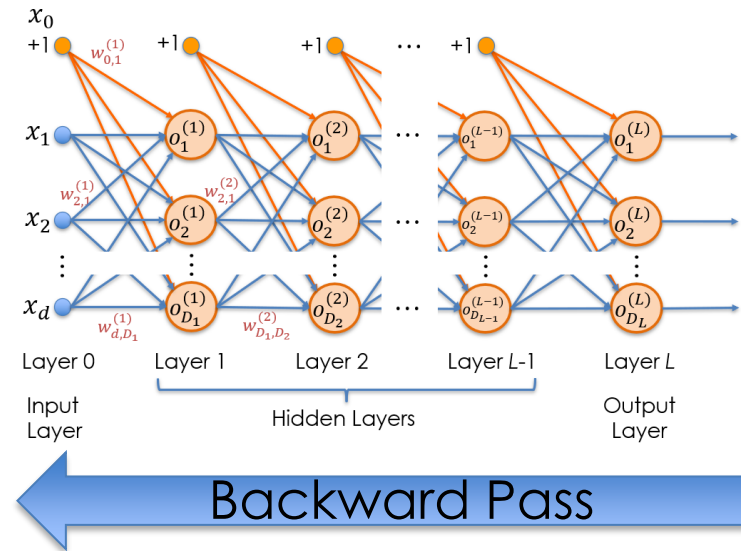
$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}}$$

$\epsilon_j^{(l)}$ (red box) $a_i^{(l)}$ (blue box)

- If hidden layer ($l < L$):

$$\frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \frac{\partial \mathcal{L}(z_1^{(l+1)}, \dots, z_{D_{l+1}}^{(l+1)})}{\partial z_j^{(l)}} = \sum_{m=1}^{D_{l+1}} \frac{\partial \mathcal{L}(z_m^{(l+1)})}{\partial z_j^{(l)}}$$

$$\begin{aligned}
 &= \sum_{m=1}^{D_{l+1}} \frac{\partial \mathcal{L}(z_m^{(l+1)})}{\partial z_m^{(l+1)}} \frac{\partial z_m^{(l+1)}}{\partial z_j^{(l)}} = \sum_{m=1}^{D_{l+1}} \epsilon_m^{(l+1)} \frac{\partial z_m^{(l+1)}}{\partial o_j^{(l)}} \frac{\partial o_j^{(l)}}{\partial z_j^{(l)}} \\
 &= \sigma^{(l)'}(z_j^{(l)}) \sum_{m=1}^{D_{l+1}} \epsilon_m^{(l+1)} w_{j,m}^{(l+1)}
 \end{aligned}$$



Multilayer Perceptron (MLP)

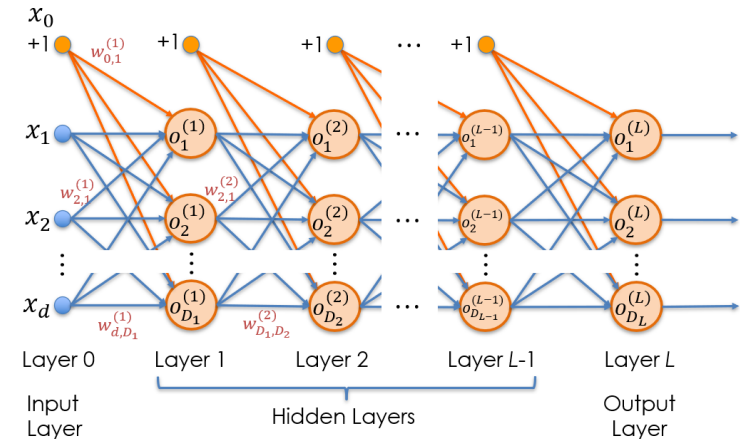
➤ Learning (W): Backpropagation

If hidden layer ($l < L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sigma^{(l)'}(z_j^{(l)}) \sum_{m=1}^{D_{l+1}} \epsilon_m^{(l+1)} w_{j,m}^{(l+1)} a_i^{(l)}$$

e.g., if $\sigma^{(l)}(z) = \max(0, z)$, what is $\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}}$?

ReLU



Multilayer Perceptron (MLP)

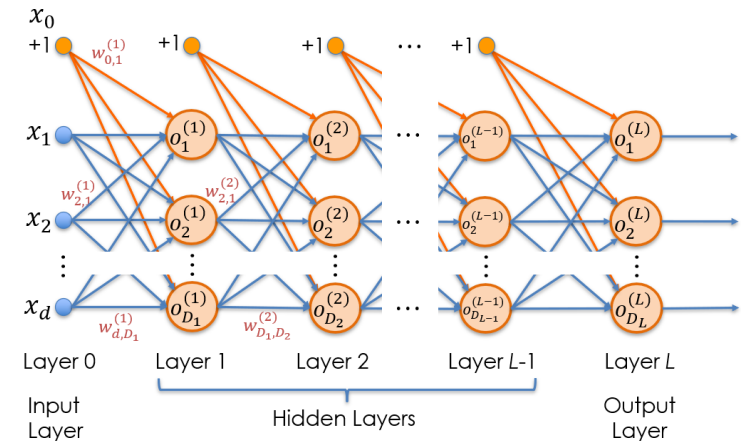
➤ Learning (W): Backpropagation

If hidden layer ($l < L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sigma^{(l)'}(z_j^{(l)}) \sum_{m=1}^{D_{l+1}} \epsilon_m^{(l+1)} w_{j,m}^{(l+1)} a_i^{(l)}$$

e.g., if $\sigma^{(l)}(z) = \max(0, z)$, what is $\frac{d\mathcal{L}}{d\mathbf{w}^{(l)}}$?

ReLU



Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

If output layer ($l = L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)}) a_i^{(L)}$$

If softmax activation, categorical cross entropy loss:

$$\frac{d\mathcal{L}}{d\mathbf{W}^{(L)}} = \mathbf{a}^{(L)} \underbrace{(\mathbf{f}_{\mathbf{W}}(\mathbf{x}) - \mathbf{y})}_{\boldsymbol{\epsilon}^{(L)}}, \quad \frac{dJ}{d\mathbf{W}^{(L)}} = \frac{1}{N} (\mathbf{A}^{(L)})^T \underbrace{(\mathbf{F}_{\mathbf{W}}(\mathbf{X}) - \mathbf{Y})}_{\mathbf{E}^{(L)}}$$

If hidden layer ($l < L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sigma^{(l)'}(z_j^{(l)}) \sum_{m=1}^{D_{l+1}} \epsilon_m^{(l+1)} w_{j,m}^{(l+1)} a_i^{(l)}$$

If ReLU activation:

$$\frac{d\mathcal{L}}{d\mathbf{W}^{(l)}} = \mathbf{a}^{(l)} (\boldsymbol{\delta}_{\mathbf{z}^{(l)}} * (\boldsymbol{\epsilon}^{(l+1)} (\mathbf{W}_{1:D_L}^{(l+1)})^T))$$

$$\frac{dJ}{d\mathbf{W}^{(l)}} = \frac{1}{N} (\mathbf{A}^{(l)})^T (\boldsymbol{\delta}_{\mathbf{z}^{(l)}} * (\mathbf{E}^{(l+1)} (\mathbf{W}_{1:D_L}^{(l+1)})^T))$$

Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

Example: A MLP of 2 layers has been constructed as:

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}) = \sigma^{(2)}\left(\left[\mathbf{1}, \sigma^{(1)}\left(\mathbf{x}^T \mathbf{W}^{(1)}\right)\right] \mathbf{W}^{(2)}\right),$$

where $\sigma^{(1)}$ is **ReLU**, and $\sigma^{(2)}$ is the **Softmax**.

$$\text{Training data: } \mathbf{X} = \begin{bmatrix} 1 & 1 & 3.0 \\ 1 & 2 & 2.5 \end{bmatrix}, \mathbf{Y} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

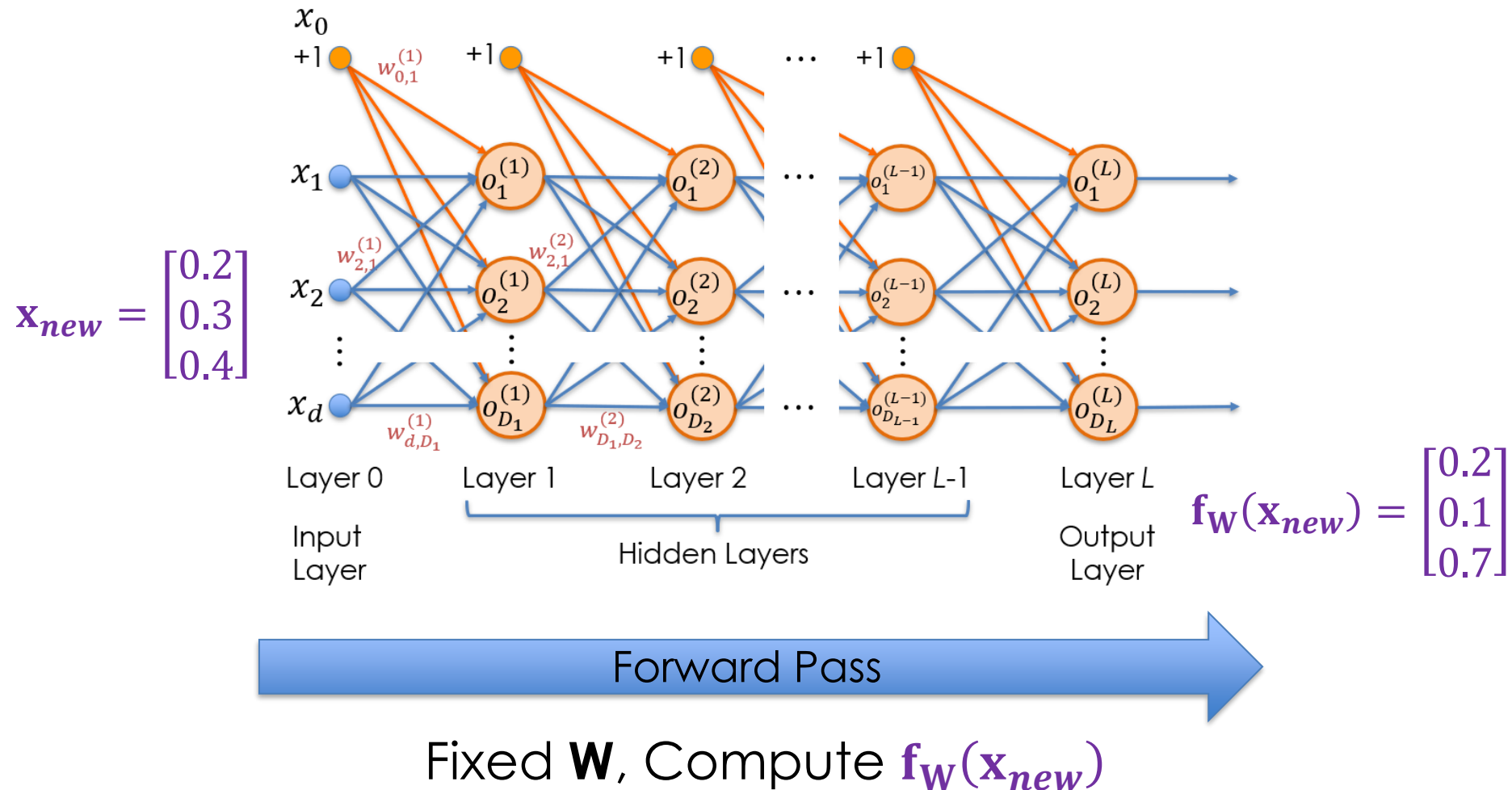
Loss function: **Categorical Cross Entropy**

$$\text{Weight initialization: } \mathbf{W}^{(1)}(0) = \mathbf{W}^{(2)}(0) = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}$$

What are $\mathbf{W}^{(1)}(1)$ and $\mathbf{W}^{(2)}(1)$ after 1st iteration of weight updates if learning rate $\eta=0.1$?

Multilayer Perceptron (MLP)

➤ Prediction (Forward Pass)



MLP for Image Classification

MLP for Image Classification

➤ Image is stored as a matrix in a computer



0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

Pixel: smallest unit of a digital image

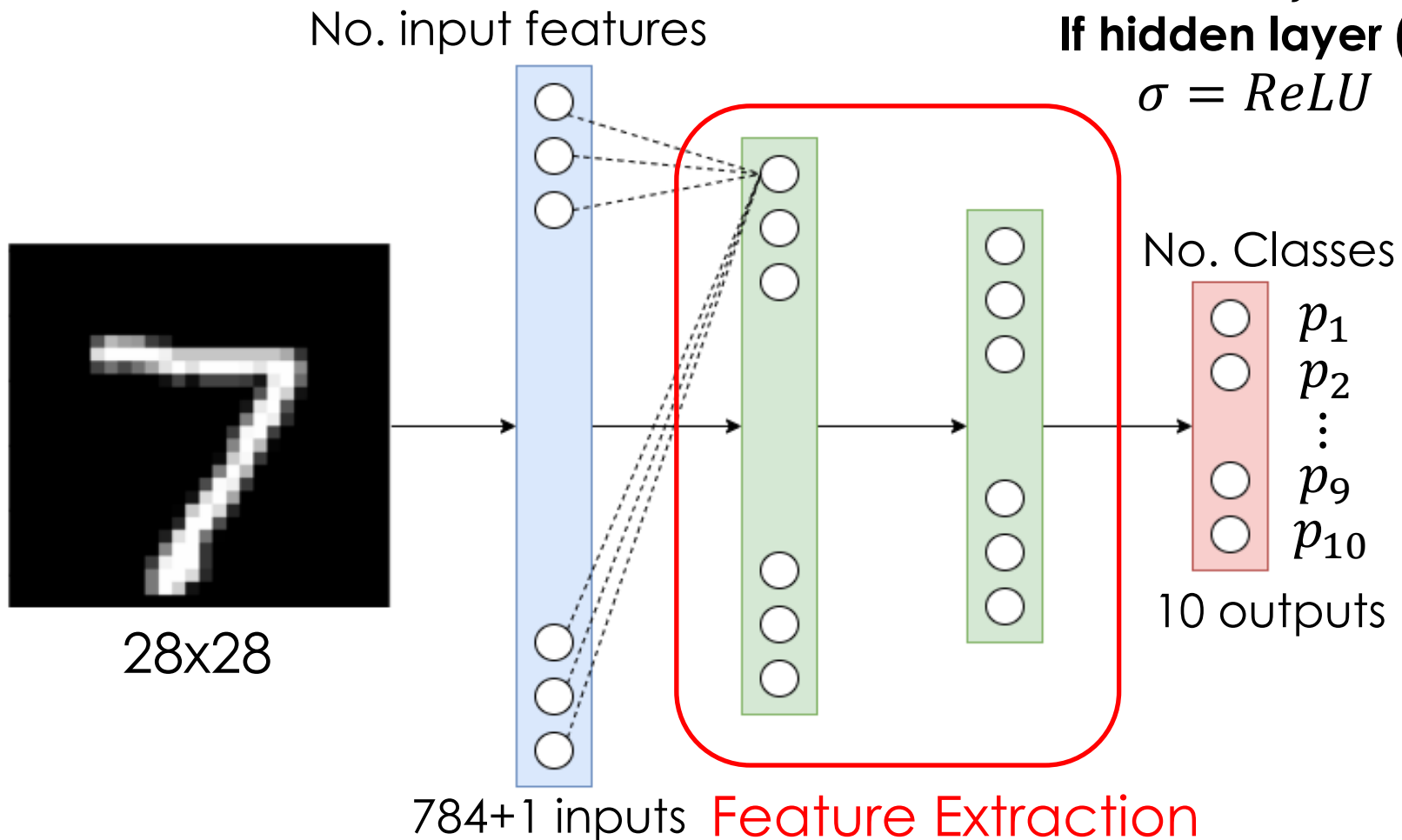
MLP for Image Classification

If output layer ($l = L$):

$$\sigma = \text{Softmax}$$

If hidden layer ($l < L$):

$$\sigma = \text{ReLU}$$

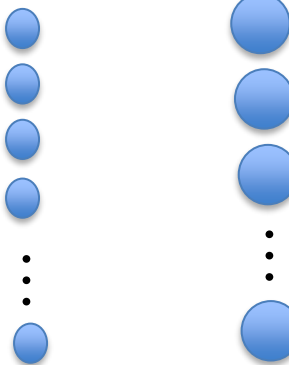


MLP for Image Classification

➤ Limitations



200x200



How many
input features?

How many parameters to
learn for the first layer?

10K neurons in the
first layer



Flattening

Summary

✓ Perceptron

✓ Multilayer Perceptron (MLP)

-Learning ($\mathbf{W}$)

1. Random Initialization of $\mathbf{W}$

2. Forward Pass (fixed $\mathbf{W}$)

3. Backward Pass (update $\mathbf{W}$)

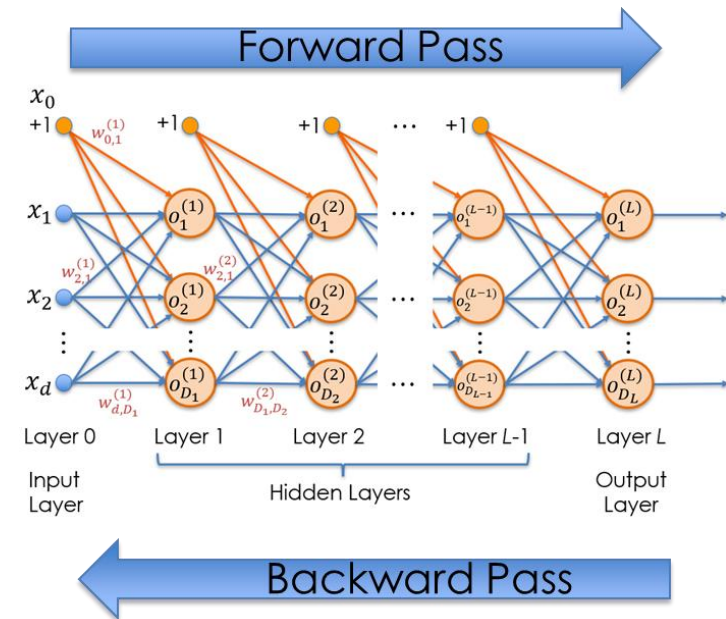
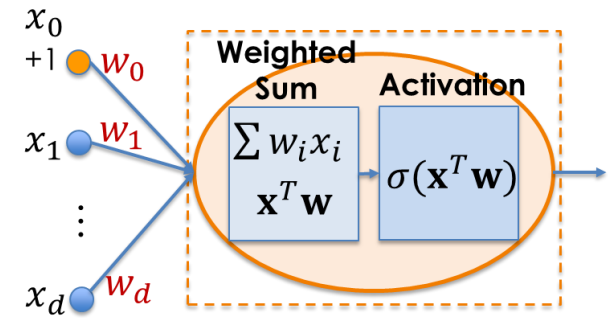
$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \nabla_{\mathbf{W}} J(\mathbf{W}_{old})$$

-Prediction (Forward Pass)

$$\mathbf{f}_{\mathbf{W}}(\mathbf{x}_{new})$$

✓ MLP for Image Classification

Limitations



THE END